Towards Utilizing Cached Context for Faster and Smarter Code Agents

Frank Chen

CMU-CS-25-139

August 2026

Computer Science Department
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

Thesis Committee
Prof. Rashmi K. Vinayak, Chair
Prof. Zhihao Jia

Submitted in partial fulfillment of the requirements for the degree of Master of Science.

Keywords: LLM, Agent, Cache, Retrieval Augmented Generation, Software Engineering

Abstract

Recent advances in Large Language Models (LLMs) have enabled the development of coding agents that can autonomously perform tasks such as code generation, debugging, patch validation, etc. Industry-proprietary systems (e.g., Colab AI, Claude Code, Cursor Agent) and open-source frameworks (e.g., Gemini Cli, SWE-Agent, AutoCodeRover, Agentless) have demonstrated both practicality and popularity in real-world software engineering workflows. Despite these successes, existing agentic frameworks face two common challenges: providing accurate and complete context information and ensuring low-latency patch generation under heavy workloads. Although prior work has proposed partial solutions addressing either context retrieval or latency, little attention has been paid to the joint optimization of both aspects. Ideally, joint optimization should enhance both performance and speed without incurring additional cost, which is particularly critical in modern fast-paced, iterative software engineering environments. This thesis integrates existing state-of-the-art approaches to these challenges, specifically RepoGraph for context retrieval on the frontend and CacheBlend for position-independent KV cache reuse on the backend, and contributes a set of enhancements that adapt CacheBlend's reference implementation to agentic conditions: completion of the position-correction path for chunks reused at a new offset, batched allocation and bounded eviction in the storage backend, and a per-request profiler. We evaluate accuracy of the frontend across multiple LLMs under realistic code-agent scenarios, and measure the backend's latency against vLLM with prefix caching on a trace generated by the frontend under the same benchmark. Cache reuse preserves patch quality, but it is profitable only in a high-hit-ratio regime and above a prompt-length floor: across our trace only 17.7% of requests are faster, and applied to all traffic the cache layer is a net slowdown. Profiling attributes this to a synchronous KV store on the critical path whose cost grows with prompt length, so a request pays for what it writes even when it reads nothing. We therefore propose an end-to-end design in which a routing controller treats the cache layer as optional per request. Overall, the findings argue that position-independent reuse belongs in a code agent, but applied selectively rather than by default.

Acknowledgments

Contents

1 Introduction **1**

2 Background **4**
2.1 Position-Independent KV Cache Reuse 4
2.2 Repository-Aware Context Retrieval 5
2.3 Motivation for Integration 7

3 System Design **8**
3.1 Cached Context 8
3.2 System Architecture 9
3.2.1 Backend: API Server with CacheBlend 9
3.2.2 Adapter Module 10
3.2.3 Frontend: RepoGraph + Agentless 10
3.3 Backend Enhancements 11
3.3.1 Position-Independent Reuse 11
3.3.2 Memory Allocation Under Concurrency 12
3.3.3 Debugging and Profiling Infrastructure 13
3.4 Frontend Integration Details 15
3.4.1 Context Truncation 15
3.4.2 Local Store for Precomputed Data 15

4 Evaluations **16**
4.1 Latency Evaluation for KV Reuse 16
4.1.1 Setup 16
4.1.2 Results 18
4.1.3 Analysis: Small-Request Speedup Variance 20
4.1.4 Case Study: Routing Requests Around the Cache 25
4.2 Accuracy Evaluation for Context Retrieval 31
4.2.1 Setup 31
4.2.2 Results 32

5 Conclusion **34**
5.1 Summary of Contributions 34
5.2 Summary of Findings 35
5.3 Limitations and Future Work 35

Bibliography **37**

List of Figures

3.1 End-to-end system architecture. 9

3.2 Share of prompt tokens served from cache across the agent trace, under alignment-dependent reuse (previous) and position-independent reuse (current). 12

3.3 Example profiler output: per-operation latency timeline during a CacheBlend-enabled serving session (chunk lower bound 128 tokens). Each point is a single profiled operation, color-coded by type. The clustering of `retrieve_layer` (orange) at 25–75 ms reflects the roughly token-independent per-call retrieve cost analyzed in Section 4.1.3; the token-proportional store cost is analyzed in Section 4.1.4. 14

4.1 Request length distribution of the SWE-bench Verified-mini trace ($n = 537$). . . 17

4.2 Per-request TTFT speedup against cache hit ratio, colored by prompt length in tokens. Speedup separates by cache hit ratio, not by request size: both clusters span the full length range, and the two populations are divided by a band of hit ratios in which the served run contains no observations. Four requests above 5× are outside the plotted range; cluster statistics are computed over all requests. . 18

4.3 TTFT speedup distribution by prompt token-count bucket (outliers removed). Violin plots show the probability density; box plots mark the median and interquartile range. 19

4.4 Distribution of per-request TTFT speedup across 537 requests in the SWE-bench Verified-mini trace. 20

4.5 Chunked-prefill fragmentation rate by prompt token-count bucket. Requests are classified by the number of prefill steps their prompt occupied: non-fragmented (1 step, 4 cache-engine operations across the 4 tensor-parallel workers) or fragmented (9 steps, 36 operations). The plot covers the 421 requests belonging to these two groups; the 116 requests with other step counts are excluded, so each bar sums to 100% within the two groups. Small requests (<2k tokens) exhibit the highest fragmentation rate at 45%. 21

4.6 Per-call `retrieve_layer` cost broken down by phase group (blend vs. no-blend). GPU Onload dominates at 91% of blend's per-call time, with a 1.93× ratio over no-blend due to gap-aware KV recomputation. 22

4.7 TTFT speedup vs. total prompt tokens (log scale), colored by fragmentation class. Non-fragmented requests (blue circles) cluster tightly below breakeven for small prompts; fragmented requests (orange diamonds) exhibit the full variance spread. Both classes converge for long prompts. The plot covers the 532 requests that remain after speedup outliers are removed, so the counts in its legend are slightly below the trace-wide counts given in the text. 23

4.8 Median TTFT speedup vs. prompt length, split by cache hit ratio. Requests that hit the cache sit below breakeven from 1,000 to 3,000 tokens and well above it beyond 4,500; the shaded band marks the interval containing the single crossing. Requests that miss sit near 0.5× at every length, so no length threshold recovers them. Prompts under 1,000 tokens are excluded because the cache path's fixed cost dominates the ratio at that size. 26
4.9 Served-run cache hit ratio against measured prompt length ($n = 536$). The same two clusters and the same empty band appear as in Figure 4.2. Length is a weak predictor of which cluster a request falls into ($r = -0.21$): the per-bucket median declines with length, but well-cached requests occur at every length. 27
4.10 Per-request TTFT speedup under three routing policies. Bypassed requests are served by vanilla vLLM and sit at exactly 1.0× by construction, so they are excluded and each histogram shows only the requests still taking the cache path. The legend reports the aggregate deficit for each policy. 29
4.11 Resolve rate distributions for SWE-bench Verified-mini evaluation runs across different models. 33
4.12 Resolve rate distributions for SWE-bench Verified-mini evaluation runs across different blending methods, served model is Devstral Small. 33

Chapter 1

Introduction

Recent studies on AI coding agents have proven the efficacy of using Large Language Models (LLMs) to handle coding tasks such as code generation and debugging. To apply LLMs in daily software engineering scenarios, both industry and academia have proposed many agentic frameworks. Industry-proprietary examples include Google's Colab AI [SKS25], Claude Code [Ant25], and Cursor Agent [LIB25]. There is a continuous and rapid growth of user bases for these frameworks, reflecting their perceived utility in real-world software engineering tasks. Besides the proprietary frameworks, there are also many open-source implementations from both academia and industry, including Gemini Cli [MS25], SWE-Agent [Yan+24], AutoCodeRover [Zha+24c], and Agentless [Xia+24]. These frameworks enable existing LLMs to achieve high task success rates when prompted with clear instructions and sufficient code context.

A common characteristic of these agents is that they issue many requests to the LLM during a single task. Each request often carries a long prompt: repository structure information, multiple code snippets retrieved as context, system instructions, and the issue description. The repeated and overlapping nature of this content across requests within a code repository means a serving system that re-processes every prompt from scratch wastes a large fraction of compute on tokens it has already seen. This redundancy directly translates to high time-to-first-token (TTFT) and high cumulative cost per task. The latency cost is particularly important because agentic workflows are interactive: a user typically waits for the agent's output, and slow generation degrades developer productivity in a way that aggregates across many sessions.

The standard solution to this redundancy is KV cache reuse, where the serving engine stores intermediate attention states from previous requests and reuses them when the same tokens appear again. Modern serving engines such as vLLM [Kwo+23] and SGLang [Zhe+24] implement prefix caching, which reuses cached KV states only when prompts share an exact prefix from the start. Throughout this thesis we refer to the LLM serving system, vLLM together with its KV cache layer, as the *backend*, and to the context retrieval system that assembles the prompts as the *frontend*. In an agentic setting, calls often share partial contexts that appear after some unique tokens, leading to low cache hit rates under prefix-only matching [Kwo+23; Zhe+24]. CacheBlend [Yao+25] addresses this by enabling

position-independent KV cache reuse: the prompt is treated as a sequence of chunks, each chunk is matched against the cache independently, and a small subset of cross-attention-sensitive tokens are recomputed to restore correctness. The original CacheBlend paper demonstrates 2–3× TTFT reduction and 3–5× throughput improvement on multi-round *question-answering workloads*.

CacheBlend's original evaluations, however, did not cover the workload characteristics of code agents. Agentic prompts often contain code blocks that are reordered or partially substituted across successive requests, and they span a much wider range of cache hit ratios. A single agent task also keeps many requests in flight at once, so several requests enter the cache engine simultaneously. This thesis evaluates LMCache, the reference implementation of CacheBlend, under code-agent workloads, and our evaluation unearths several inefficiencies that this regime exposes. The mechanism that corrects positional encodings when a chunk is reused at an offset different from the one it was cached at is incomplete, so position-shifted reuse fails at runtime. The storage backend allocates one memory object at a time under a global lock, which serializes requests that carry many chunks. The KV store runs synchronously inside the model execution step, so every request pays a token-proportional offload cost inside its time-to-first-token even when it reads nothing from the cache. We propose and implement fixes for the first two inefficiencies. We identify the third and quantify its cost, but leave a redesign of the store path to future work.

This thesis presents a set of enhancements to LMCache that address these limitations, together with an end-to-end evaluation under realistic code-agent workloads. The core enhancements are on the backend serving system:

- **Position correction for non-contiguous reuse**: a cached chunk could not be reused at a position different from the one where it was first cached. We complete the mechanism that corrects positional encodings, so that a content-matched chunk can be reused at any position in a later prompt.
- **Enhanced memory allocation and locking**: the cache stores each chunk of a prompt as a separate memory object, and allocating them one at a time meant acquiring the same allocator lock once per chunk, which serialized requests that carry many chunks. We add a batched varied-shape allocator that takes the lock once per request instead of once per chunk, so lock contention no longer grows with the number of chunks. We also bound the eviction loop: when the cache is full, the allocator could retry forever if every entry is pinned by another in-flight request, so we cap the number of retries and fall back to normal computation once the cap is reached.
- **Per-request profiling infrastructure**: a thread-local profiler instruments the cache engine, storage backend, and GPU connector along the critical path, emitting structured per-operation and per-phase timing records, including per-step logs of the retrieve path, for post-hoc analysis.

To evaluate these enhancements under realistic agentic traffic, the system is wrapped in an end-to-end pipeline that includes a context retrieval frontend. Producing a realistic agentic workload requires a method for assembling the per-request prompts that a real code agent would issue, including repository structure, relevant code snippets, and an

issue description. We use RepoGraph [Ouy+25] combined with the Agentless procedural framework [Xia+24] as a concrete instantiation of the frontend. The frontend is treated as a pluggable component: any context retrieval module that produces structured prompts with explicit chunk boundaries can be substituted for it without affecting the backend design. Our frontend evaluation primarily serves to verify that the backend cache reuse does not degrade output quality and that the integrated system performs consistently across multiple LLM backends.

Our evaluation shows that the benefit of blending in this setting is concentrated rather than uniform: only 17.7% of the 536 requests in the trace complete faster with blending enabled. Whether a request wins is governed by its cache hit ratio, not by its prompt length. The trace splits into a low-hit cluster with hit ratios of at most 34.2% (n=430, median speedup 0.496×, 2.1% of requests above breakeven) and a high-hit cluster with hit ratios of at least 72.3% (n=106, median speedup 2.058×, 81.1% above breakeven). No request in the trace has a hit ratio between 34.2% and 72.3%, so we can bound the breakeven point to that interval but cannot measure it. The cost of blending is dominated by data movement: per-call cache retrieve overhead is 30.8 ms with blending against 18.0 ms without it, a factor of 1.72, and GPU onload accounts for 91% of the blend cost. On a cache miss the penalty is roughly 2× and is nearly independent of prompt length, because the KV store runs synchronously inside the time-to-first-token window and the amount of data transferred grows in proportion to the prompt. A prompt-length floor near 2,000 tokens does exist, but it applies only to requests that actually hit the cache. Cache reuse does not degrade output quality: resolve rates are comparable across blending granularities. The take-away is that position-independent reuse pays off only when the serving system can anticipate a high hit ratio, which makes admission control on the cache path more important than prompt size in code-agent workloads.

The remainder of this thesis is organized as follows. Chapter 2 provides background on KV cache reuse and CacheBlend, then on context retrieval and RepoGraph, and motivates an integrated system that supports realistic backend evaluation. Chapter 3 presents the system design, with emphasis on the backend cache engine enhancements that constitute our core contribution, followed by the frontend integration details required to complete the end-to-end pipeline. Chapter 4 reports the latency evaluation of the enhanced backend under agentic workloads, followed by the supporting accuracy evaluation of the context retrieval frontend across multiple LLM backends.

Chapter 2

Background

This chapter provides the technical background needed to motivate the system design in Chapter 3. Section 2.1 covers the latency bottleneck caused by redundant prefill computation in agentic workflows and presents CacheBlend as the state-of-the-art approach to position-independent KV cache reuse. Section 2.2 covers context retrieval as the complementary frontend concern and presents RepoGraph as the concrete retrieval module used in our integrated system. Section 2.3 explains why an integrated system is needed to evaluate the backend enhancements under realistic agentic workloads.

2.1 Position-Independent KV Cache Reuse

A primary source of latency and resource inefficiency in agentic LLM serving is redundant computation across requests. Many agentic frameworks send a large number of sequential requests to the LLM during a single task. The prompt in each request typically includes bulk context information and template tokens. When no cache reuse is in place, the model re-processes these tokens from scratch every time. For instance, an agent might retrieve a large snippet of code, feed it into the model to propose a fix, and then in a subsequent step feed much of the same snippet again while testing or refining the fix. Each of these calls recomputes attention over the repeated context tokens. This redundant computation is a shared inefficiency across many agentic workflows.

At the core of this inefficiency is the key-value (KV) cache in transformer models, which stores intermediate attention states of processed tokens so that each new token need not recompute attention over the entire sequence. The cache normally resets between separate API calls. To reuse computation across calls, several optimized LLM serving engines have introduced caching and scheduling strategies. vLLM [Kwo+23] improves throughput by batched execution and paged memory management of the KV cache, treating it as pageable memory. It can serve multiple generation requests in parallel and merge requests with identical prefixes so that the shared prefix is computed only once. Even vLLM, however, only reuses prefix computations when those prefixes exactly align from the start of the prompt [Kwo+23; Zhe+24]. In an agent scenario, calls often share partial contexts

that appear after some unique tokens, leading to low cache hit rates. SGLang [Zhe+24] introduces RadixAttention, storing caches in a radix tree keyed by prefixes and scheduling requests to maximize cache hits, though it too relies on exact prefix matching. Another approach, Prompt Cache [Gim+24], allows explicitly marking reusable prompt segments and precomputing their KV states. While this yields substantial latency reductions on long-document QA tasks, it requires manual template setup and cannot adapt if templates change.

Cache optimization for LLM inference has recently taken a leap forward with CacheBlend [Yao+25]. CacheBlend's key innovation is enabling *position-independent* reuse of KV caches. It treats the prompt as composed of chunks, checks for each chunk whether it has stored KV state, and when reassembling chunks selectively recomputes a small subset of tokens. The recomputation target is the set of HKVD (High-KV-Deviation) tokens, which are tokens whose cached KV state would otherwise deviate too far from what the model would have computed if the prompt were processed from scratch. Recomputing these tokens restores the effect of cross-attention on the forward-attention matrix. The observation that HKVD tokens tend to remain deviated through different layers also makes it possible to select and recompute them efficiently. On the multi-round QA workloads used in the original paper, this blending enables much higher cache hit rates and yields a reported 2–3× reduction in time-to-first-token and 3–5× throughput improvement without degrading output quality. CacheBlend was demonstrated effective in multi-round QA scenarios through datasets such as 2WikiMQA and Musique [Ho+20; Tri+22].

CacheBlend's reference implementation, LMCache, leaves several aspects of agentic workloads unaddressed. First, the mechanism that corrects positional encodings when a chunk is reused at an offset different from where it was originally cached is incomplete, so position-shifted reuse does not work in practice. As a concrete consequence, if a code snippet appears at positions [120, 180] in one request and at positions [400, 460] in the next, the cached KV state from the first request cannot be applied directly to the second, because the rotary embeddings were computed for the original positions and need to be corrected for the new ones. Second, the storage backend allocates one memory object per cache chunk, acquiring the in-memory CPU storage lock on each allocation. For a prompt with N chunks this serializes N lock acquisitions, which becomes a bottleneck under high concurrency. Third, the eviction loop on a full cache retries indefinitely, which can deadlock when all entries are pinned by concurrent lookups on other threads. Finally, the KV store runs synchronously inside the model execution step, so a request pays an offload cost proportional to its prompt length inside its own time-to-first-token, even when it reads nothing from the cache. Chapter 3 presents the modifications introduced to address the first three. The fourth is identified by our evaluation in Chapter 4, which quantifies its cost and shows that it dominates the aggregate result.

2.2 Repository-Aware Context Retrieval

While cache reuse on the backend addresses latency, the quality of the output the agent produces depends on the context that is included in the prompt. This is a separate concern

from the backend, but a complete end-to-end system requires some method for assembling per-request prompts that contain the right code context for a given task. We briefly review the state of context retrieval here because the frontend used in our evaluation builds on this prior work.

Early code LLM agents demonstrated strong capabilities on isolated coding tasks, but they worked from incomplete or irrelevant context when the task spanned a large repository. Frameworks such as SWE-Agent [Yan+24], AutoCodeRover [Zha+24c], and Agentless [Xia+24] fed the model only limited snippets of a codebase or generic summaries. These agents reached the rest of the codebase by searching keywords, files, and directories, together with a scrolling mechanism for file viewing. SWE-Agent, for example, exposed three commands: `goto`, `scroll_up`, and `scroll_down`, which let the agent move a constant-sized context window to any line of the target file [Yan+24]. This linear view of the repository left the agent with false assumptions on code structure and implementation details, leading to unstable patch quality. Providing an agent with only a function implementation but not the definitions of its referenced symbols omitted crucial information and hurt the quality of patches. Supplying too much code instead overwhelmed the model with irrelevant details and interfered with its reasoning [Zha+24a; Zha+24b]. This tension motivated the repository-aware retrieval work that followed.

Several lines of work have addressed this tension. SWE-Search [Ant+25] integrates Monte Carlo Tree Search to explore the solution space, and CodePlan [Bai+23] tracks incremental code changes with dependency analysis. CoSIL [Jia+25] dynamically constructs a module-level call graph during the agent's search, expanding the graph by following function calls and pruning irrelevant branches. RepoGraph [Ouy+25] takes a complementary approach by pre-building a complete repository graph before the agent starts working. It constructs a repository-level code graph that captures relationships between code elements across different files: each class, method, or function is a node, and the edges connect definitions to their usage or dependency. During a setup phase, RepoGraph parses the source files to identify where each symbol is defined and where it is referenced, links each definition to its uses, and stores the resulting graph for fast querying. When the agent needs context for a particular function or error, it queries this graph to retrieve all related definitions and usage chains. RepoGraph has been shown to consistently boost the resolve rate of Agentless and SWE-Agent when paired with capable backends such as GPT-4o or Claude-3.5-Sonnet [Ouy+25].

We use RepoGraph as the concrete context retrieval module in our integrated system. The choice is pragmatic rather than essential to the contribution: RepoGraph produces structured, chunk-friendly prompts that expose natural cache boundaries, and its open-source implementation is straightforward to integrate. The adapter design in Section 3.2.1 keeps the frontend interface narrow, so any alternative retrieval module that produces prompts with explicit chunk boundaries (for example a more recent agentic-search-based system) can be substituted without affecting the backend.

2.3 Motivation for Integration

The original CacheBlend evaluations used multi-round question-answering datasets in which prompts share structured text chunks across turns. While these workloads validate the position-independent reuse mechanism in principle, they leave open the question of how the system performs under code-agent workloads, where prompts can consist of long code context, repository structure information, and reordered context blocks across requests. To evaluate this in a realistic setting, we wrap LMCache (with the backend enhancements introduced in Chapter 3) in an end-to-end agentic pipeline that uses RepoGraph and Agentless as the frontend to generate realistic prompts. The backend enhancements are the central technical contribution; the integration with the frontend serves as the evaluation harness that exposes the cache engine to representative agentic traffic.

A second motivation for the integrated system is a quality control point. Position-independent cache reuse trades a small amount of computed-from-scratch fidelity for latency savings when the cache hits, and it is important to confirm that the resulting outputs remain usable in practice. Our frontend evaluation in Section 4.2 addresses this question directly by comparing the resolve rate of the integrated system under different blending granularities against a no-blending baseline.

The mechanism that ties the backend cache engine to the frontend prompts is what we refer to as *Cached Context*. Rather than treating a prompt as a single unit that must line up end to end with an earlier one, the frontend splits each prompt it produces into separate cache units at the boundaries of individual context objects: one code snippet, one repository structure block, or one template section. The backend then matches and reuses each unit on its own, so reuse no longer depends on whole prompts agreeing. Cached Context exploits the modular and repetitive structure of code-agent context, which consists mostly of repository structure information and function or class definition blocks, alongside prompt templates whose tokens recur on every request. Section 3.1 formalizes this concept, and Chapter 3 presents the full system design.

Chapter 3

System Design

This chapter presents the design of the end-to-end system. We first introduce the Cached Context concept that organizes prompts so that the backend cache engine can exploit reuse opportunities (Section 3.1). We then describe the system architecture, with emphasis on the backend serving path that constitutes our central contribution and a brief account of the frontend that feeds it (Section 3.2). Section 3.3 details the backend enhancements that adapt CacheBlend's reference implementation to agentic workloads. Section 3.4 covers the implementation details on the frontend side that are needed to complete the end-to-end pipeline.

3.1 Cached Context

Cached Context is a prompt-organization concept rather than a caching mechanism on its own. The key idea is to chunk each prompt at a per-context-object granularity, where a context object corresponds to a code snippet containing a symbol definition, a repository structure tree, or another semantically coherent unit of context. The actual caching is performed by the backend, which combines CacheBlend's content-addressed segment lookup with the position correction enhancement introduced in Section 3.3.1. The two together make a cached chunk reusable regardless of its position in subsequent prompts, which is what we refer to as *position-independent caching*.

To illustrate why the choice of chunk granularity matters, consider the following example. Suppose the agent constructs a prompt after context retrieval, consisting of four parts: system instructions, repository structure, issue description, and a block of relevant code snippets. A naive chunking strategy would treat this prompt as a single unit or divide it into four coarse blocks. With Cached Context, we further decompose the context blocks into finer-grained chunks. The relevant code snippets block, for example, might contain definitions for multiple symbols retrieved by RepoGraph, where each symbol definition is now its own cache chunk.

Now consider a subsequent prompt in the same workflow, but with slightly different localization and a partly different set of code snippets, while most symbols and other

blocks remain the same. Under conventional prefix caching, the cache can be reused only when the overlapping content appears identically from the start of the prompt. With Cached Context paired with the backend's position-independent caching, each overlapping symbol definition is a cache hit regardless of where it appears in the new prompt. This is particularly valuable in agentic workflows, where the order and combination of context elements frequently varies across requests.

3.2 System Architecture

Figure 3.1 illustrates the overall architecture. The design prioritizes modularity, allowing each component to be developed and tested independently. The remainder of this section walks through the backend and frontend in that order.

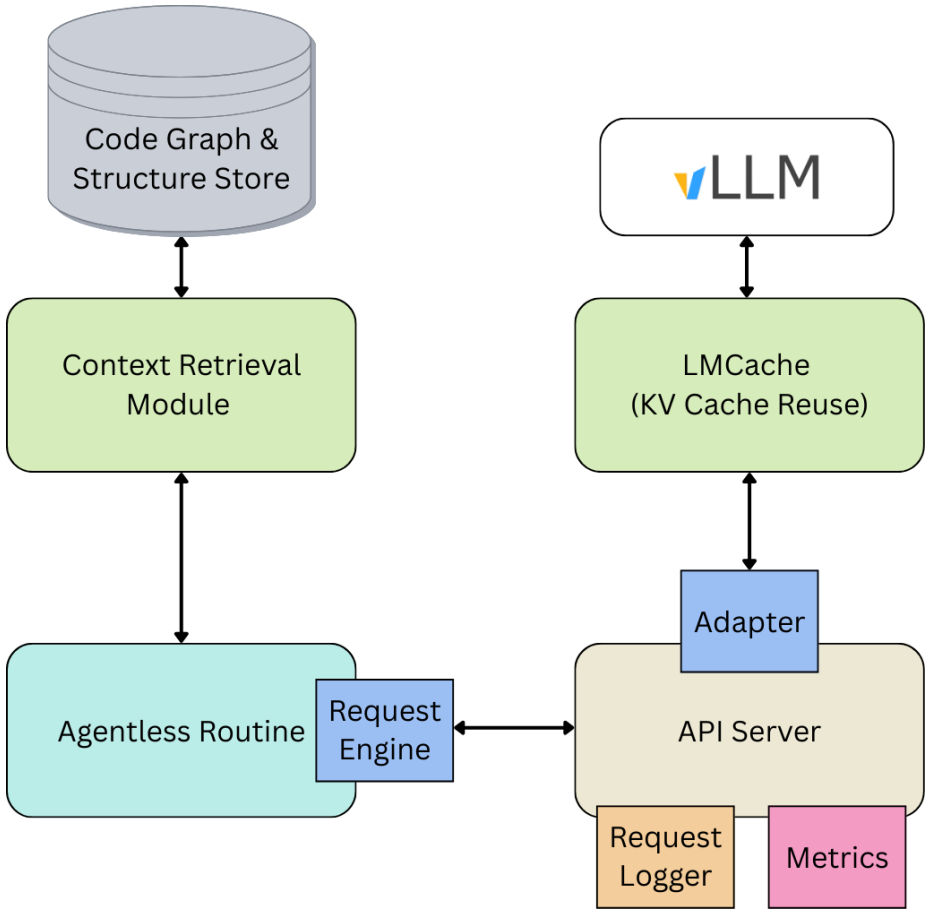

Figure 3.1: End-to-end system architecture.

3.2.1 Backend: API Server with CacheBlend

The backend consists of an API server that serves as the entry point for frontend requests. The server hosts the LLM inference engine (vLLM) with LMCache integration, which

implements the CacheBlend algorithm for position-independent KV cache reuse. When a request arrives, the server performs the following steps:

1. Parse the chat messages from the request.
2. Pass the messages through the adapter module (described below) for tokenization and chunk extraction.
3. Forward the processed prompt to the LLM engine with CacheBlend enabled.
4. Return the generated response to the client.

The backend exposes a chat-completion interface compatible with the OpenAI API format. This means any compatible frontend can drive the backend without modification beyond annotating context-object boundaries in its prompt templates.

3.2.2 Adapter Module

The adapter module serves as the bridge between the frontend prompt format and the backend cache engine. Its responsibilities include:

1. **Chunk extraction**: identifying and retrieving individual context objects from the incoming chat messages.
2. **Tokenization**: encoding each chunk using the model's tokenizer, including correct handling of models with custom tokenization schemes (e.g., Devstral).
3. **Chunk stitching**: reassembling the tokenized chunks to produce the final prompt token sequence consumed by the LLM engine.

The adapter's design supports flexible chunking granularity based on the frontend prompt template and user preference. If chunking is per symbol, the frontend adds a delimiter after each code snippet fetched during code graph exploration. If a coarser granularity is desired (e.g., per context block), delimiters can be placed only between major sections of the prompt. This flexibility makes the system robust to various workload patterns and to differing optimal chunk sizes.

3.2.3 Frontend: RepoGraph + Agentless

We use RepoGraph as the frontend context retrieval algorithm, integrated with the Agentless procedural framework. Agentless follows a three-step routine:

1. **Localize**: given an issue description, identify the files and code regions most likely to require modification.
2. **Repair**: generate candidate patches for the localized regions.
3. **Rerank**: evaluate and rank the candidate patches, selecting the most promising one for submission.

RepoGraph enhances the localization phase by providing repository-aware context. Rather than relying solely on file-level search, Agentless can query RepoGraph's code graph to retrieve symbol definitions, call chains, and structural information. This integration ensures that the LLM receives semantically complete context. For example, when

localizing a bug in a function, the model also receives definitions of the symbols referenced by that function.

To enable Cached Context, we modify Agentless's prompt generation routine to make context object boundaries explicit, which exposes chunk boundaries to the backend adapter without affecting the semantic content of the prompt. The frontend is otherwise treated as a pluggable component: any retrieval module that produces prompts with explicit chunk boundaries can replace RepoGraph + Agentless without changes to the backend.

3.3 Backend Enhancements

This section presents the backend enhancements that adapt CacheBlend's reference implementation (LMCache) to the workload characteristics of code agents. Each addresses a capability the agentic regime demands and the question-answering regime does not: reuse of cached state at shifted positions, allocation that tolerates many chunks per prompt and many prompts at once, and enough visibility into the cache path to attribute cost within it. The last of these is presented together with the request logging and replay mechanism, since the two jointly provide the measurement basis for the latency evaluation in Chapter 4.

3.3.1 Position-Independent Reuse

Reusing a cached chunk at a position other than the one it was first computed at requires two capabilities, and they are independent of each other. The first is *finding* the chunk: the cache must be addressed by content, so that a block of context is recognized as the same block wherever it appears in a prompt. The second is *using* it: the retrieved state must be re-anchored to the position the chunk now occupies. A system that provides only the first can locate reusable state but cannot apply it, and its reuse collapses back to the prefix case it was meant to improve on.

CacheBlend supplies the first capability. Each chunk is keyed by a hash of its own tokens rather than by its offset, so lookup is already position-independent. If one prompt carries the ordered chunks $[C_1, C_3, C_4]$ and a later prompt carries $[C_2, C_3, C_4]$, the cache recognizes C_3 and C_4 in the second prompt even though a different chunk now precedes them.

The second capability is what our system adds. Modern transformers encode token position through rotary embeddings, which are folded into the key and value vectors at the time they are computed. Cached state therefore carries the positions it was born with. A chunk first seen at positions $[120, 180]$ holds embeddings for those positions, and dropping it unmodified into a prompt where it now spans $[400, 460]$ would present the model with attention state describing a sequence that does not exist. Correctness requires rotating the cached keys from their original positions to their current ones as they are loaded, which means the cache must remember where each chunk was when it was stored, and the load path must apply that rotation.

We carry the original positions alongside each cached chunk and re-anchor it on load. The consequence is that reuse no longer depends on alignment. Returning to the example,

C_3 and C_4 are not only found in the second prompt but usable in it, and they remain usable if the agent reorders them, so $[C_1, C_3, C_4]$ followed by $[C_2, C_4, C_3]$ still reuses two chunks of three. A chunk that happens to land at its original offset is rotated by zero and behaves exactly as before.

This matters for code agents specifically because their prompts are rearrangements of a slowly changing pool of context. Successive requests in a single task retrieve overlapping sets of symbol definitions and repository structure, but the localization step reorders them and substitutes a few between turns. Under alignment-dependent reuse, a single inserted or dropped block at the front of a prompt invalidates everything after it. Under position-independent reuse, that same edit costs only the block it touched.

Figure 3.2 shows what this is worth on the agent trace used throughout Chapter 4. Measured as the share of all prompt tokens served from cache rather than recomputed, reuse rises from 14.7% to 23.0%, an increase of 8.3 percentage points, or a little over half again as much reuse as alignment-dependent matching achieves. The gain comes entirely from chunks that were already present in the cache and already being found there, but that could not previously be applied because they had moved.

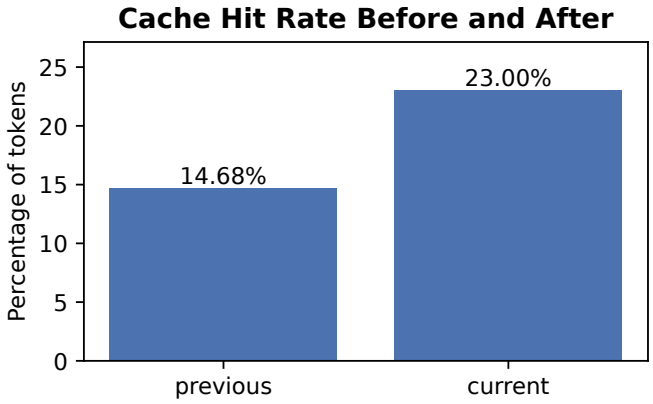

Figure 3.2: Share of prompt tokens served from cache across the agent trace, under alignment-dependent reuse (previous) and position-independent reuse (current).

3.3.2 Memory Allocation Under Concurrency

The cache holds its hot tier in main memory, staging chunks there before they are loaded back onto the GPU. Two properties of agentic traffic stress that tier in ways that question-answering workloads do not: a single prompt is divided into many chunks rather than a few, and several requests from the same agent task are typically in flight at once. Both concern how allocation behaves under contention rather than how much memory is available.

Allocation granularity. Storing a prompt means obtaining memory for every one of its chunks, and chunks vary in size because context objects do. If the allocator is entered once per chunk, the cost of admission to the cache scales with how finely the prompt was divided, and the serialization that protects the shared memory pool is paid over and over for what is logically one operation. This penalizes exactly the prompts that Cached Context

is designed to produce. We instead admit a prompt's chunks as a single transaction: the allocator is told all the sizes at once, reserves them together, and makes room in one pass if capacity is short. Contention then scales with the number of concurrent requests rather than with the number of chunks inside them, and a prompt can no longer be admitted halfway before a competing request reclaims the space it still needs.

Reclamation under pinning. Making room means evicting older entries, and an entry currently being read by another request cannot be evicted. When many requests are resident at once, it is possible for every candidate to be pinned, at which point an allocator that simply waits for space will wait forever while holding the pool that would let others make progress. The condition is rare but it is a livelock rather than a slowdown, so it does not resolve on its own. We treat exhaustion as an ordinary outcome instead: reclamation gives up after a bounded number of attempts, the store is abandoned, and the request proceeds by computing its own prefill. Losing one opportunity to cache is a far cheaper failure than stalling the engine, and the cache converges back to normal operation as soon as the pinned entries are released.

3.3.3 Debugging and Profiling Infrastructure

Diagnosing why a given request did or did not benefit from cache reuse requires visibility at two levels: what the cache engine spent its time on inside a request, and what sequence of requests the agent produced in the first place. We implement one subsystem for each, and together they form the measurement basis for the latency evaluation in Chapter 4.

Per-request profiling. To enable fine-grained latency analysis of the cache engine under realistic workloads, we implement a per-request profiling module that instruments the critical path through the adapter, cache engine, storage backend, and GPU connector. The profiler is organized around two abstractions: *operations*, which correspond to logical units of work (e.g., a single request's retrieve pass), and *phases*, which subdivide an operation into timed intervals (e.g., token database lookup, per-layer storage retrieval, per-layer GPU onload).

The profiler uses thread-local storage to maintain a per-thread operation stack, ensuring that timing records from tensor-parallel workers do not bleed across threads. Each phase is recorded with its wall-clock duration, parent operation identifier, and thread ID. On completion, the profiler emits per-operation summaries in JSONL format, enabling post-hoc aggregation and visualization.

Figure 3.3 shows an example timeline produced by the profiler during a CacheBlend-enabled serving session. Each point represents a single profiled operation, plotted by wall-clock time on the x-axis and measured latency on the y-axis, and color-coded by operation type. The `retrieve_layer` operations (orange) dominate the per-step cost on the read path, typically clustering between 25 and 75 ms with occasional spikes above 100 ms. The `store_layer` operations (green) cover the synchronous GPU-to-CPU offload of newly computed KV states, which Section 4.1.4 identifies as the dominant cost whenever the

cache misses. The `adapter.start_load_kv` markers (red, near the baseline) indicate the lightweight entry point into the cache path. This visualization enables rapid identification of latency outliers and temporal patterns that would be invisible in aggregate statistics.

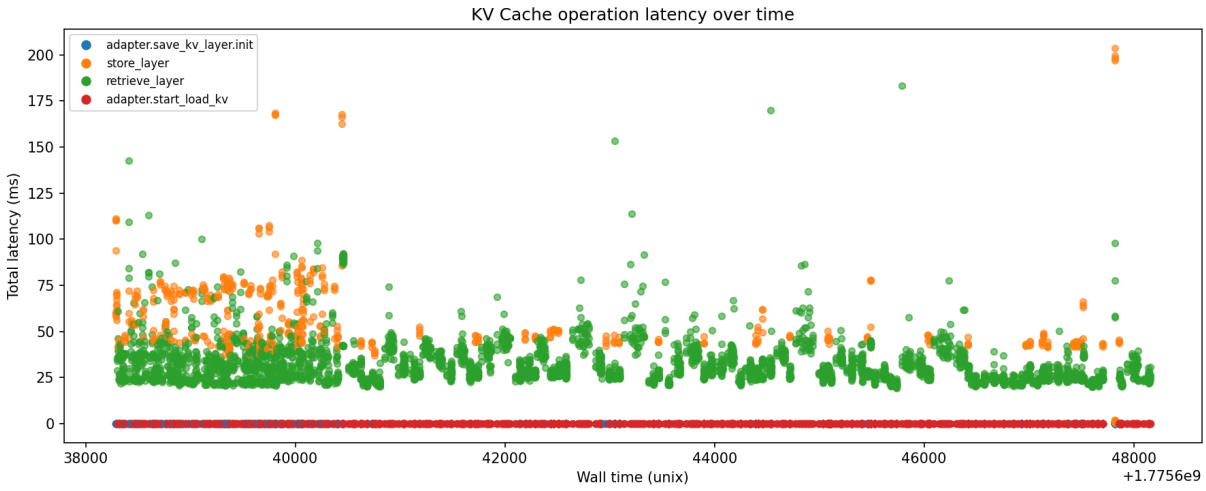

Figure 3.3: Example profiler output: per-operation latency timeline during a CacheBlend-enabled serving session (chunk lower bound 128 tokens). Each point is a single profiled operation, color-coded by type. The clustering of `retrieve_layer` (orange) at 25–75 ms reflects the roughly token-independent per-call retrieve cost analyzed in Section 4.1.3; the token-proportional store cost is analyzed in Section 4.1.4.

This instrumentation is what makes performance anomalies visible at all. It revealed both the source of the speedup variance among small requests (Section 4.1.3) and the synchronous KV store path that dominates the penalty on cache misses (Section 4.1.4). Neither would have been visible from end-to-end TTFT measurements alone.

Request logging and replay. To support debugging and reproducible experimentation, the API server includes a request logging module. During a server session, all incoming requests and their corresponding responses are captured and stored as a local trace file.

This transparency enables inspection of which stage during the Agentless routine may have caused a performance bottleneck. For example, if localization prompts consistently exhibit low cache hit rates, the prompt template can be adjusted to improve context alignment across requests.

In addition, the server supports a **replay** functionality that parses requests from any trace file and simulates the workload internally. This feature enables:

- Evaluation on captured traces from real agentic workloads without re-running the full agent pipeline.
- Modularized testing through synthetic small traces that exercise specific caching scenarios.
- A/B comparison of different caching configurations on identical workloads.

These capabilities are essential for the latency evaluation methodology in Chapter 4. The replay mechanism lets us drive the backend with a fixed, deterministic trace under both blend and no-blend configurations, eliminating the variability that would otherwise come from re-running the agent loop end-to-end.

3.4 Frontend Integration Details

Beyond the core architecture, several implementation details on the frontend side improve the system's robustness and the realism of the prompts that reach the backend.

3.4.1 Context Truncation

For complex coding tasks involving multiple files and nested dependencies, the frontend can exceed the model's token limit when constructing a prompt. Originally, prompts were naively truncated at the end when the token limit was reached. This approach frequently results in important instructions (such as output format requirements or validation criteria) being omitted from the prompt. Even when the LLM receives sufficient context to understand the problem, it may underperform due to missing instructions.

Our solution leverages the model's tokenizer to calculate the token length of each context block and the prompt template. Before constructing the prompt, the last context block is truncated to exactly fit the remaining token budget, so that the critical pieces on the template stay intact and instructions are passed to the LLM as desired. This improves the reliability of the generated patches when the context is long.

3.4.2 Local Store for Precomputed Data

Computing the code graph and repository structure from scratch for every task instance incurs significant latency overhead. RepoGraph's graph construction involves parsing all source files using `tree-sitter`, identifying symbol definitions and references, and building the dependency graph. For large repositories, this process can take several minutes.

To address this, we introduce a local store that caches these large data structures. The store is keyed by repository ID and commit hash, ensuring that cached data remain valid across tasks targeting the same codebase state. This optimization is particularly valuable when processing multiple issues against the same repository (common in benchmark evaluations like SWE-bench) or when the same repository is revisited across different agent sessions.

Chapter 4

Evaluations

The evaluation comprises two parts. We first characterize the latency behavior of the enhanced CacheBlend backend under agentic workloads, which is the primary evaluation of the central contribution of this thesis, and close that section by recommending a routing policy that follows from the measurements (Section 4.1). We then evaluate the frontend context retrieval algorithm across multiple LLM backends and verify that the backend cache reuse does not degrade output quality (Section 4.2).

4.1 Latency Evaluation for KV Reuse

To assess whether CacheBlend is profitable under agentic workloads, we measure the per-request time-to-first-token (TTFT) ratio between the blend and no-blend configurations on traces generated by the RepoGraph frontend.

4.1.1 Setup

We evaluate the latency characteristics of CacheBlend by replaying agentic traces generated by the RepoGraph + Agentless frontend against vLLM with and without LMCache integration. All experiments use Devstral Small served with 4-way tensor parallelism. For the blend condition, LMCache is integrated with CacheBlend enabled at a minimum chunk size of 128 tokens. For the baseline condition, the same model is served without LMCache, using only vLLM's native prefix caching. Both conditions process the identical trace via the replay mechanism described in Section 3.3.3.

Metric

We adopt TTFT as our primary latency metric, defined as the elapsed time from request arrival to the emission of the first output token. TTFT directly captures the prefill-phase cost, which is the phase that cache reuse is designed to accelerate. For each request, we compute the *TTFT speedup* as the ratio of baseline TTFT (no blending, no LMCache) to

blend-mode TTFT. A speedup greater than 1.0 indicates that cache reuse reduced prefill latency; a value below 1.0 indicates overhead-induced regression.

Datasets

We replay the full set of prompts produced by RepoGraph + Agentless on SWE-bench Verified-mini [Mar25]. The resulting trace comprises 537 requests spanning prompt lengths from 119 to over 100,000 tokens and cache hit ratios from 0% to 99.2%.

Figure 4.1 shows how request length is distributed across the trace. Half the requests fall between 1,792 and 6,978 tokens, the median is 5,042, and a thin tail extends to 119,808. This shape matters for interpreting the latency results, because it means most requests are large enough to amortize a fixed per-request overhead but not large enough for prefill cost to dominate everything else.

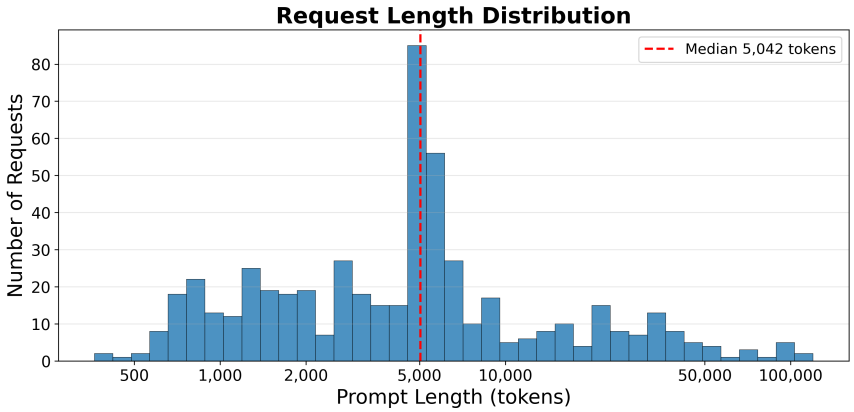

Figure 4.1: Request length distribution of the SWE-bench Verified-mini trace ($n = 537$).

Model

All experiments use Devstral Small, which is also the open-source model used in the accuracy evaluation (Section 4.2). Using a single model in both evaluations isolates the effect of the backend cache reuse from any model-to-model variation.

Configurations

We compare the following two configurations:

- **Baseline** (vLLM with prefix caching): vLLM with its built-in prefix caching enabled, which reuses KV states only for exact prefix matches. No LMCache integration.
- **Experimental** (vLLM + LMCache with CacheBlend): vLLM with LMCache integration and CacheBlend enabled, allowing position-independent KV cache reuse across non-contiguous segments.

4.1.2 Results

Speedup as a Function of Cache Hit Ratio and Prompt Length

Figure 4.2 plots TTFT speedup against cache hit ratio for every request in the trace, with each point colored by request length. The picture is organized almost entirely by the horizontal axis. Both clusters span the full length range, from a few hundred tokens to over 100,000, so request size does not determine which side of breakeven a request lands on.

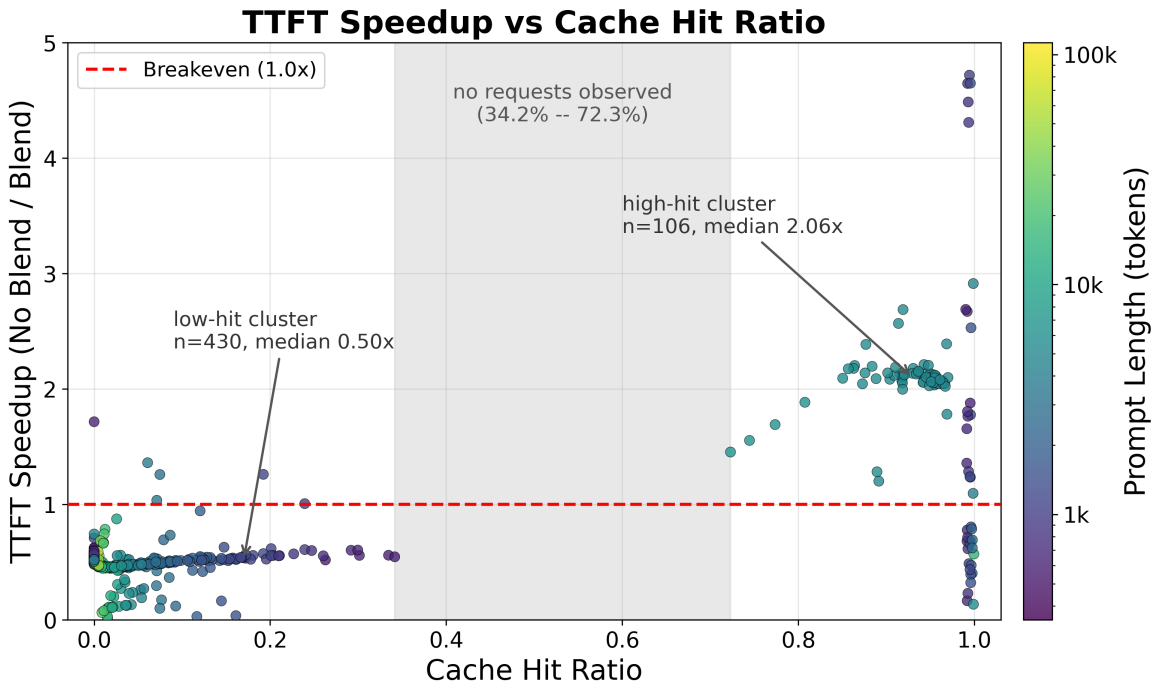

Figure 4.2: Per-request TTFT speedup against cache hit ratio, colored by prompt length in tokens. Speedup separates by cache hit ratio, not by request size: both clusters span the full length range, and the two populations are divided by a band of hit ratios in which the served run contains no observations. Four requests above 5× are outside the plotted range; cluster statistics are computed over all requests.

Cache hit ratio separates the two outcomes cleanly. The trace contains no request with a hit ratio between 34.2% and 72.3%, so the observations fall into two disjoint clusters. In the low-hit cluster (hit ratio at most 34.2%, $n = 430$) the mean speedup is 0.602 and the median 0.496, and only 2.1% of requests exceed 1.0×. In the high-hit cluster (hit ratio at least 72.3%, $n = 106$) the mean speedup is 1.935 and the median 2.058, and 81.1% of requests exceed 1.0×. Across the full trace, 17.7% of requests come out ahead under blending. Because the two clusters are separated by a band in which we observed nothing, the breakeven crossing cannot be located from this trace. It can only be bounded: it lies somewhere between 34.2% and 72.3% hit ratio.

Figure 4.3 breaks the same data down by prompt-length bucket, and shows that prompt length is a much weaker predictor than hit ratio. The median speedup sits near 0.5× in every bucket. What changes across buckets is the spread rather than the center: the <2k

and 5–10k buckets carry long upper tails that reach well above breakeven, while the 10–30k and >30k buckets are tightly concentrated below it. Long prompts are therefore the most *predictable* case, but they are predictably slower under blending in this trace, not faster. The requests that benefit are those with high cache hit ratios, and in this trace those are not the longest prompts.

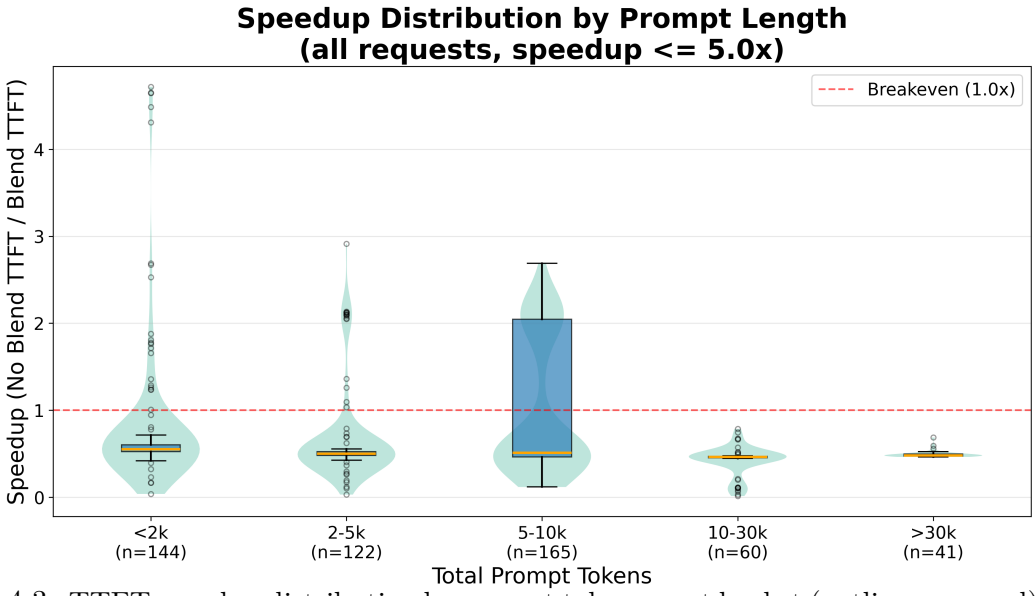

Figure 4.3: TTFT speedup distribution by prompt token-count bucket (outliers removed). Violin plots show the probability density; box plots mark the median and interquartile range.

Notably, this variance among small requests persists even when restricting to well-cached requests. Among requests with cache hit ratio at least 50%, the <2k bucket retains a spread from 0.3× to 9.5×, while the 5–10k bucket collapses to a tight 1.9–2.5× band. A high cache hit ratio alone does not guarantee fast performance for small requests. Two further factors govern the outcome: prompt length, whose effect on well-cached requests is examined in Section 4.1.4, and scheduler-induced fragmentation, examined in Section 4.1.3.

Speedup Distribution

Figure 4.4 shows the distribution of per-request TTFT speedup across the full trace (mean 0.866, median 0.509). The distribution is bimodal, and the two modes correspond directly to the two hit-ratio clusters identified above. The primary mode sits at a median of 0.494× and holds the 441 requests that fall below breakeven, which is 82.3% of the trace. The secondary mode sits at a median of 2.095× and holds the remaining 95 requests. The separation is driven by cache hit ratio rather than by prompt length: as Figure 4.7 makes clear, requests below breakeven form a dense band near 0.5× that persists across the entire prompt-length range, from a few hundred tokens to over 100,000, while the requests above breakeven are concentrated in two groups near 1,000 and 5,000 tokens rather than at the long-prompt end.

This distribution is a property of the trace as much as of the system. The SWE-bench Verified-mini trace is derived from real-world Django issues, and the localization decisions

RepoGraph makes on this repository produce a hit-ratio distribution in which most requests share little content with their predecessors. A workload whose successive prompts overlap more heavily would move mass into the high-hit cluster, where the measured speedup is roughly 2× for prompts above the length floor identified in Section 4.1.4 and still below 1.0× beneath it. We did not measure such a workload here, so this remains a hypothesis rather than a result.

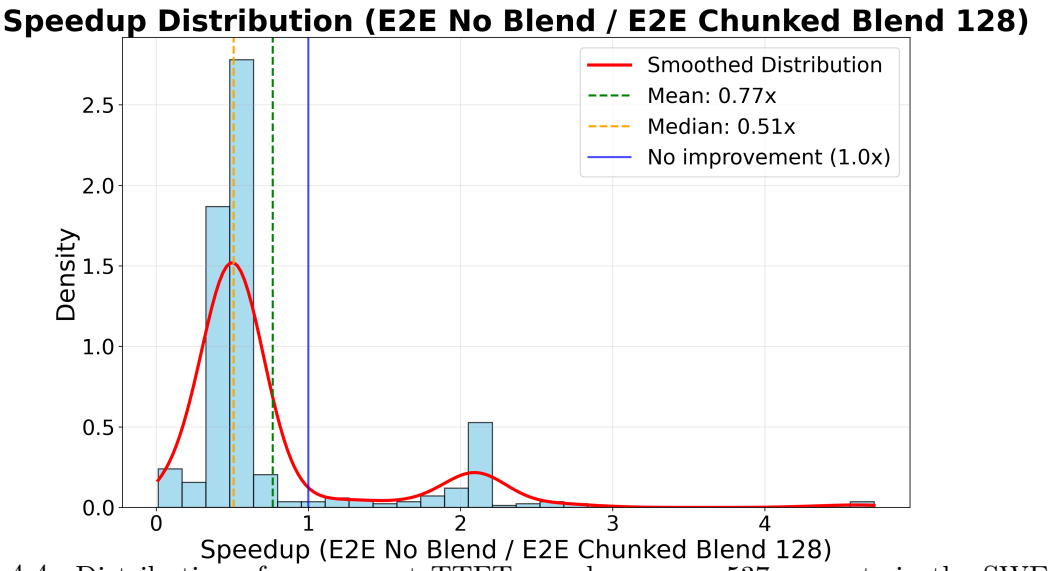

Figure 4.4: Distribution of per-request TTFT speedup across 537 requests in the SWE-bench Verified-mini trace.

4.1.3 Analysis: Small-Request Speedup Variance

The speedup plot reveals that requests with fewer than 5,000 tokens exhibit significantly higher speedup variance than longer requests at comparable hit ratios. Using the per-request profiling infrastructure described in Section 3.3.3, we trace the root cause to an interaction between vLLM's chunked-prefill scheduler and LMCache's per-step invocation pattern.

Scheduler-Induced Fragmentation

We use *fragmentation* throughout this section in a specific sense: a request is **fragmented** when the serving engine does not process its prompt in one go, but splits it across several scheduling steps. The term refers to the splitting of a single prompt's prefill work over time, not to memory fragmentation or to any partitioning of the cache itself.

This splitting is a normal consequence of how vLLM schedules work. Its chunked-prefill scheduler works from a per-step token budget, `max_num_batched_tokens` (16,384 in our configuration), that is shared across every request in flight. Sequences already generating output consume part of that budget, so a long prompt, or a prompt that arrives while other requests are decoding, receives only a fraction of the budget and has its remaining tokens deferred to later steps. We call each scheduling step in which part of a request's

prompt is processed a *prefill step*. A request handled in a single prefill step is therefore non-fragmented, and a request spread over several is fragmented.

The profiler counts cache-engine operations rather than steps, because LMCache's retrieve path is what it instruments. That path is invoked once per prefill step on each tensor-parallel worker, and these experiments use 4-way tensor parallelism, so a request's operation count is exactly four times its prefill-step count: one step produces 4 `retrieve_layer` operations and nine steps produce 36. Across the 537-request trace the counts concentrate into two dominant groups. 322 requests (60%) complete in a single prefill step, and 99 requests (18%) are spread over nine. The remaining 116 requests (22%) take other step counts, so the distribution has two strong modes rather than being strictly bimodal.

The specific value nine is not a constant of the system. No configuration setting fixes it. It emerges from the steady-state load of this particular benchmark run: with a roughly stable population of concurrently decoding sequences, a newly arrived prompt receives a roughly fixed share of the token budget per step and therefore takes a roughly fixed number of steps to finish. A different arrival rate or concurrency level would place the second mode at a different step count, though the underlying split between prompts that arrive under load and prompts that arrive between batches would persist.

Figure 4.5 cross-tabulates the fragmentation rate against prompt length. Small requests (<2,000 tokens) are fragmented at a 45% rate, which is 3.4× the rate of 5–10k requests (10%) and the highest of any bucket. This susceptibility arises because short prompts fill a smaller portion of `max_num_batched_tokens`, leaving room for the scheduler to pack decode steps alongside them and defer their remaining tokens to subsequent batches.

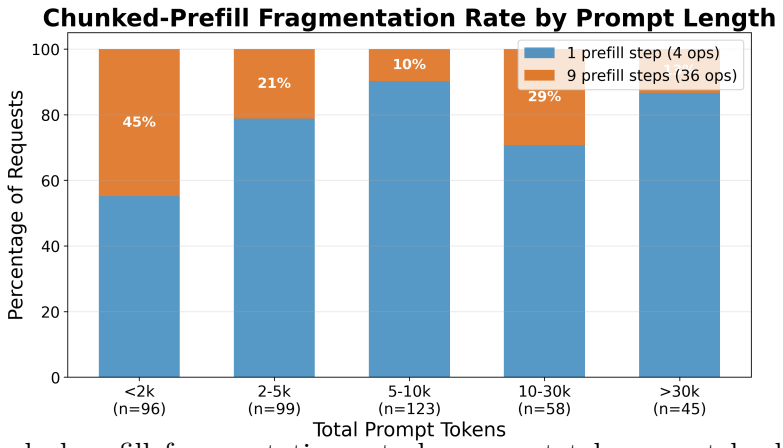

Figure 4.5: Chunked-prefill fragmentation rate by prompt token-count bucket. Requests are classified by the number of prefill steps their prompt occupied: non-fragmented (1 step, 4 cache-engine operations across the 4 tensor-parallel workers) or fragmented (9 steps, 36 operations). The plot covers the 421 requests belonging to these two groups; the 116 requests with other step counts are excluded, so each bar sums to 100% within the two groups. Small requests (<2k tokens) exhibit the highest fragmentation rate at 45%.

Per-Step Retrieve Overhead

Each invocation of the cache engine's retrieve path triggers a chain of operations whose per-call cost is roughly independent of the number of tokens in that step. Figure 4.6

presents the measured per-call cost broken down by phase group, aggregated over 4,852 blend and 3,092 no-blend `retrieve_layer` calls. Three phases dominate:

1. **Token database lookup** (0.7 ms blend, 0.3 ms no-blend): walks the token sequence to identify cached chunks.
2. **Storage retrieval** (2.0 ms blend, 3.1 ms no-blend): fetches each layer's KV cache from the CPU storage backend.
3. **GPU onload** (28.1 ms blend, 14.6 ms no-blend): copies KV data into vLLM's paged buffer and, in blend mode, executes gap-aware recomputation.

GPU onload accounts for 91% of blend's per-call time and exhibits a 1.93× ratio over no-blend. The ∼13.5 ms gap stems from CacheBlend's blender, which selects the top 15% most-divergent token positions per layer and recomputes their KV state to restore cross-attention accuracy. The overall mean per-call overhead is 30.8 ms in blend mode versus 18.0 ms without blending (a 1.72× ratio).

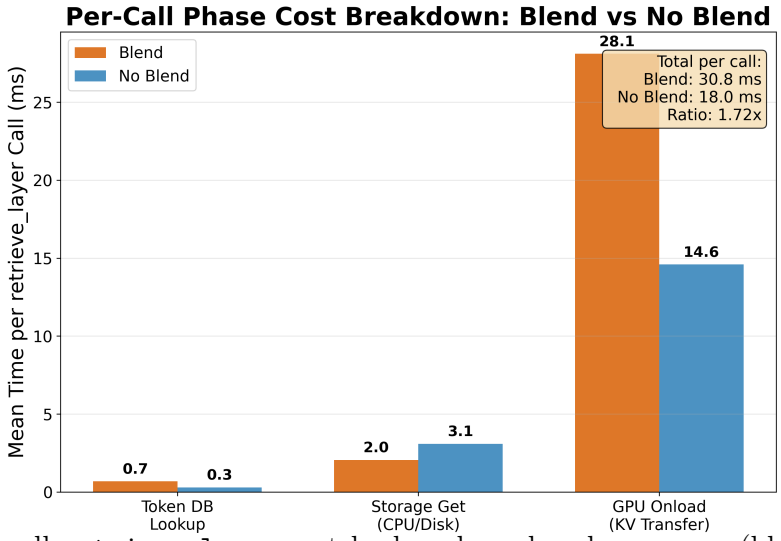

Figure 4.6: Per-call `retrieve_layer` cost broken down by phase group (blend vs. no-blend). GPU Onload dominates at 91% of blend's per-call time, with a 1.93× ratio over no-blend due to gap-aware KV recomputation.

Because this per-call cost does not shrink when a step carries fewer tokens, fragmenting a request across more steps multiplies it. A request spread over 9 prefill steps pays roughly $9 \times 30.8 = 277$ ms of retrieve overhead in wall-clock terms, against a baseline TTFT of only ∼114 ms for straight prefill of a 1,000-token prompt. (The profiler emits one record per tensor-parallel worker, so the summed record time is four times this figure; the workers execute concurrently, and only the per-step cost enters wall-clock latency.) The same 277 ms is a negligible addition to a 50,000-token request whose baseline TTFT is several seconds.

This scaling argument applies to the *retrieve* path alone, and on its own it would predict that long prompts fare well. They do not. Section 4.1.4 shows why: the dominant cost is not retrieve but the synchronous KV *store*, whose transferred volume is linear in prompt length and therefore grows in step with the baseline. Retrieve overhead explains

why fragmented short requests are erratic; the store cost explains why requests of every length settle near 0.5× when the cache misses.

Fragmentation as the Source of Variance

The preceding analysis explains why small requests are slower *on average*, but does not fully account for the extreme *variance* observed in the <5k token regime. Figure 4.7 resolves this by plotting speedup against total tokens on a log scale, with each point colored by its fragmentation class: non-fragmented (1 prefill step, 4 operations) versus fragmented (9 prefill steps, 36 operations).

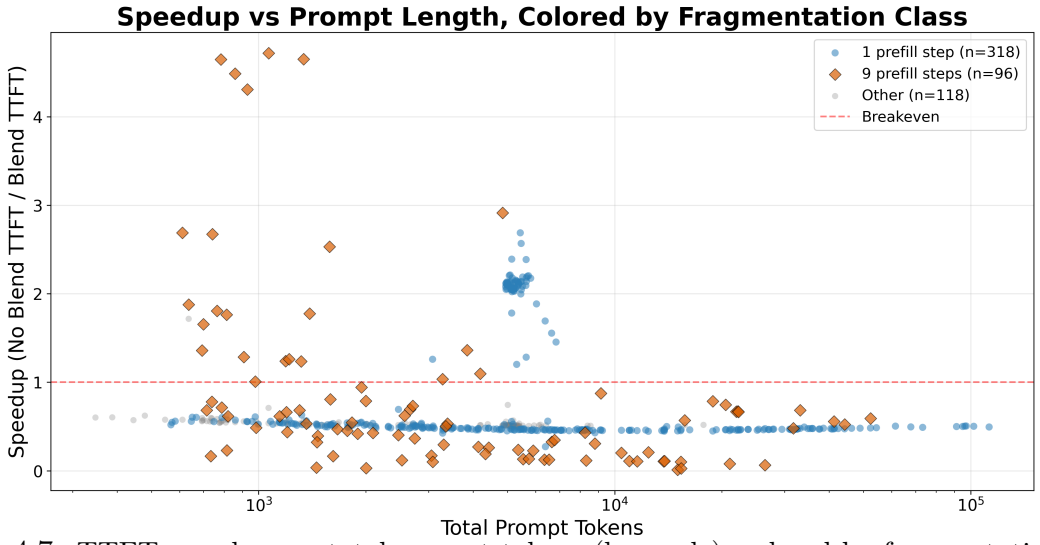

Figure 4.7: TTFT speedup vs. total prompt tokens (log scale), colored by fragmentation class. Non-fragmented requests (blue circles) cluster tightly below breakeven for small prompts; fragmented requests (orange diamonds) exhibit the full variance spread. Both classes converge for long prompts. The plot covers the 532 requests that remain after speedup outliers are removed, so the counts in its legend are slightly below the trace-wide counts given in the text.

Two distinct populations emerge among small requests:

1. **Non-fragmented small requests** (blue, single-step) cluster tightly at ∼0.54× speedup for the <2k bucket (mean 0.538, std 0.031). Blend overhead is consistent and always exceeds the compute savings for these short prompts. This population is predictable but consistently below breakeven: each request pays a single step of 30.8 ms overhead against a baseline TTFT that is itself only ∼60–200 ms.
2. **Fragmented small requests** (orange, nine-step) exhibit the full spread from near 0× to 9.5× for the <2k bucket (mean 1.604, std 1.786). The variance is driven by the interaction between the number of scheduler steps and the cache hit pattern across those steps. Some fragmented requests encounter favorable cache timing where cached content is warm from earlier steps in the same scheduling window, while others pay the full per-step overhead with minimal cache benefit.

The key insight is that **fragmentation is both the source of the worst underperformers and the best over-performers** among small requests. The scheduler's

batching decision at request arrival time determines which fragmentation class a request falls into, and this decision depends on the instantaneous batch composition rather than on the request's own cache state. Two identically sized, identically cached requests can land on opposite sides of the speedup distribution based solely on whether other decode requests happened to be in flight at the moment of arrival.

For large requests (>10k tokens), both fragmentation classes converge to the same band near 0.5×, because the retrieve overhead is a small fraction of per-step compute regardless of how many steps the scheduler chose. Fragmentation therefore explains the *variance* in the short-prompt regime specifically. It does not explain why the long-prompt requests in this trace sit at 0.5×, which follows from their low cache hit ratios combined with the token-proportional store cost analyzed in Section 4.1.4.

Implications

The fragmentation effect is not a fundamental limitation of CacheBlend's reuse model but rather a consequence of how the per-step invocation pattern interacts with the scheduler's batching decisions. Three mitigation strategies are under consideration:

(a) A *minimum token-per-step threshold* that skips cache retrieval for micro-batched steps, directly addressing the pathological case at the adapter level. This is the lowest-risk fix: it can be implemented in the adapter's per-request loop and tuned via the per-call overhead profile (30.8 ms blend, 18.0 ms no-blend).

(b) A *prompt-length-aware bypass* that routes requests away from the blend path based on prompt size. Section 4.1.4 examines this rule in detail and finds that a lower floor is worth little on its own, recovering only 3% of the aggregate deficit, while an upper threshold that bypasses long prompts recovers most of what length-based routing can achieve. Neither addresses the underlying problem, which is that length does not predict whether a request will hit.

(c) A *coalesced retrieval mode* that accumulates token ranges across prefill steps and issues a single batched lookup, eliminating the overhead multiplier. This is the most principled long-term solution but requires a non-trivial redesign of the adapter–cache-engine interface, since the current layerwise pipeline interleaves layer loads with the model's forward pass.

Approaches (a) and (b) are low-complexity and can be applied immediately, but both address the fragmentation overhead rather than the larger issue of requests that miss the cache entirely. Section 4.1.4 returns to this point and proposes a routing controller that treats the cache layer as optional per request.

The picture that emerges is that the cache path is profitable under specific conditions rather than universally. That makes the decision of which requests to send through it a design question in its own right, which the remainder of this section takes up.

4.1.4 Case Study: Routing Requests Around the Cache

The results above establish that only 17.7% of requests in our trace are faster under blending, and that whether a request wins is governed by its cache hit ratio rather than its size. This raises a practical design question: since the cache path is not universally profitable, can a request be routed *before* it reaches LMCache, so that requests unlikely to benefit go straight to vanilla vLLM? This subsection is a case study of that question: it examines what a routing controller would need to decide, and what the measured data says each candidate decision rule is actually worth. The question is worth asking precisely because cache reuse costs nothing in output quality, as Section 4.2 goes on to show, which leaves latency as the only axis on which the routing decision has to be made.

Is There a Prompt-Length Floor?

The natural first rule is a prompt-length floor. The intuition is that the cache path carries a startup cost that a short prompt cannot amortize, so below some length the lookup cannot pay for itself even in the best case. The data offers only partial support for this intuition, and establishing what support there is requires care about how prompt length is measured.

Testing it requires per-request token counts taken from the serving log rather than inferred from latency. The distinction matters: speedup is itself a ratio of baseline latency, so inferring prompt length from that same baseline would make length and speedup share a term and manufacture a correlation between them. Against measured token counts the correlation between length and speedup among well-cached requests is $r = -0.001$, whereas inferring length from baseline latency yields $r = +0.63$. All lengths reported here are measured.

Figure 4.8 plots median TTFT speedup against prompt length, separately for requests that hit the cache (hit ratio above 70%) and requests that effectively miss it (hit ratio below 5%). Prompts under 1,000 tokens are excluded, because at that size the cache path takes a near-constant 0.16 s and the speedup ratio is set by how the vanilla baseline happened to be scheduled rather than by prompt size.

Among the requests that hit, prompt length separates two regimes. Between 1,000 and 3,000 tokens they sit below breakeven, with a median of 0.80× in the lower bucket and 0.62× in the upper, and fewer than half beating the baseline. Beyond 4,500 tokens they sit well above it, at a median of 2.10× with 98% beating the baseline. The median crosses breakeven once, at roughly 2,967 tokens, within an interval of 2,659 to 4,183 tokens that reflects how coarsely the region is sampled.

That sampling is the main caveat on the crossing. Of the 85 well-cached requests at or above 1,000 tokens, 64 fall in a narrow band between 4,837 and 5,811 tokens, and only four lie between 2,000 and 4,500. The curve therefore interpolates across the region where the crossing occurs rather than measuring through it, and it is comparing two fairly distinct populations of requests rather than tracking one population as it grows. The single well-cached request above 9,000 tokens scores 0.57×, which points the other way, though one request settles nothing. We therefore treat the crossing as evidence that a length effect

exists among well-cached requests, not as a precise threshold.

The bucket below 1,000 tokens is an exception that is best read as noise rather than signal. Its median speedup is 1.66×, but individual values run from 0.17× to 9.24×. At these sizes the blend path takes a near-constant ∼0.16 s, so the ratio is governed entirely by how the vanilla baseline happened to be scheduled, and six of the nineteen requests in that bucket record a baseline above 0.4 s, which is far more than a sub-1,000-token prefill should cost. We therefore do not treat that bucket as evidence for or against a floor.

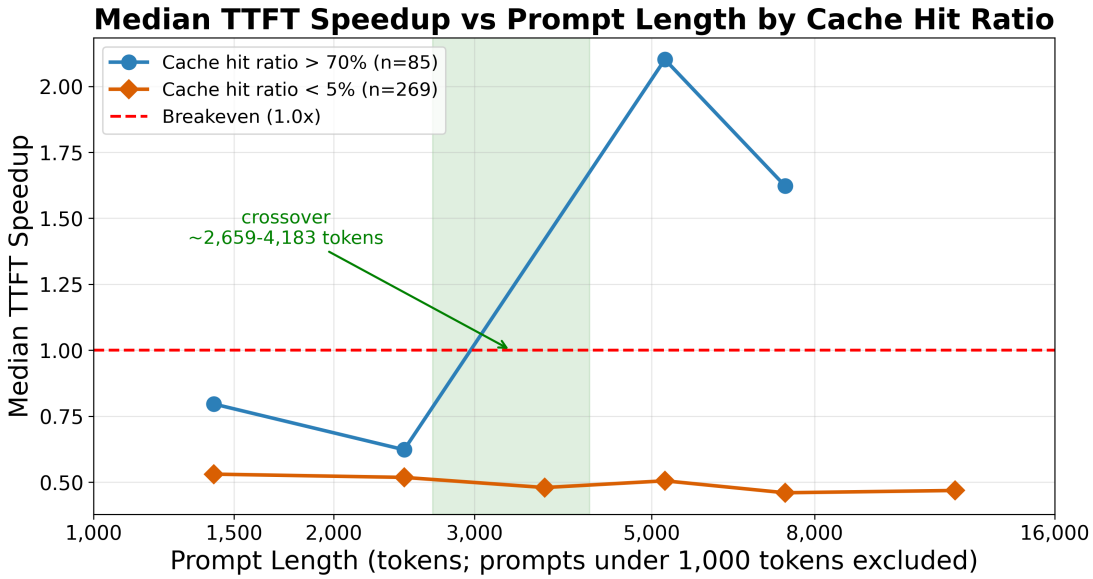

Figure 4.8: Median TTFT speedup vs. prompt length, split by cache hit ratio. Requests that hit the cache sit below breakeven from 1,000 to 3,000 tokens and well above it beyond 4,500; the shaded band marks the interval containing the single crossing. Requests that miss sit near 0.5× at every length, so no length threshold recovers them. Prompts under 1,000 tokens are excluded because the cache path's fixed cost dominates the ratio at that size.

For the requests that miss, the picture is entirely different. Their median speedup is 0.46–0.57× in every length bucket from under 1,000 tokens to over 12,000 tokens, and essentially none beat the baseline at any length. The penalty for a cache miss is close to scale-invariant: enabling LMCache roughly doubles TTFT whether the prompt is short or long. A length floor therefore cannot help this population, because there is no length at which they stop losing.

This exposes the limitation of the rule. A length floor is only useful to the extent that length predicts whether a request will hit. Figure 4.9 plots the two against each other, using the measured token counts and the same served-run hit ratios as Figure 4.2. The two clusters and the empty band between them appear here as well, which is the point: hit ratio is close to binary on this workload, and the question is whether length tells us which side a request will land on. It does so only weakly, at $r = -0.21$. The median hit ratio declines with length, from 20% below 1,000 tokens to 2.6% in the 6,000 to 10,000 range and 0.4% above 30,000, but well-cached requests appear at every length. Knowing a prompt's length narrows the odds without determining the outcome.

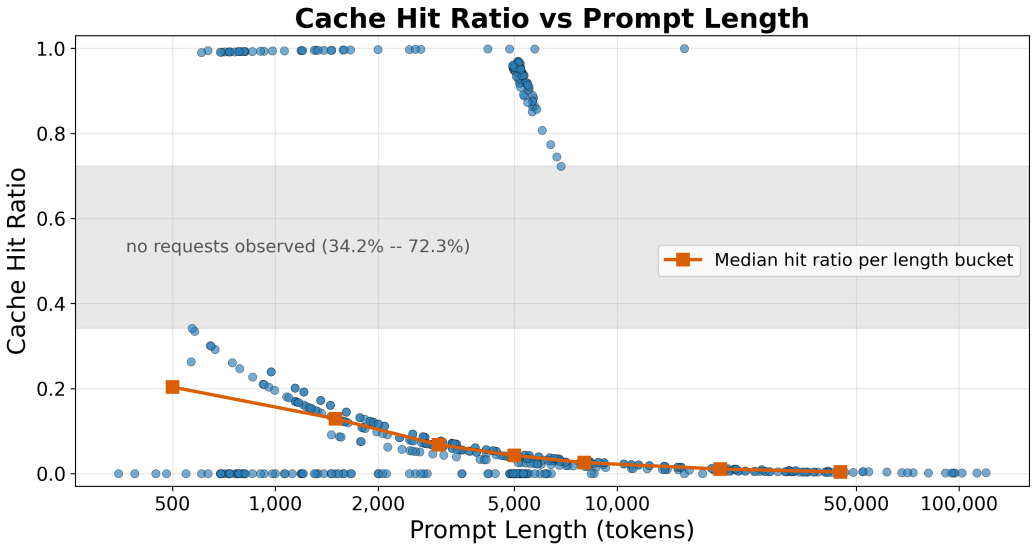

Figure 4.9: Served-run cache hit ratio against measured prompt length ($n = 536$). The same two clusters and the same empty band appear as in Figure 4.2. Length is a weak predictor of which cluster a request falls into ($r = -0.21$): the per-bucket median declines with length, but well-cached requests occur at every length.

The declining median is what the upper gate in Section 4.1.4 exploits. Beyond 6,000 tokens the median hit ratio falls to 2.6% and keeps dropping, reaching 0.4% above 30,000, so bypassing the long tail gives up little reuse. This is a property of how this agent assembles long prompts rather than of the caching mechanism, and it would need rechecking on a workload whose long requests overlap more.

Why the Miss Penalty Does Not Shrink With Length

The scale invariance follows from where the cost is incurred. On a miss, the retrieve path finds nothing and vLLM performs its normal prefill, so the lookup itself is cheap. The expense is on the *store* side. The connector's `wait_for_save` runs the entire GPU-to-CPU offload of the newly computed KV synchronously, and vLLM calls it inside `execute_model` before logits are computed and before the sampler runs. The offloaded volume is exactly linear in the number of stored tokens. Every byte of that transfer therefore lands inside TTFT, and it grows in proportion to the prompt, which is why the ratio rather than the absolute penalty is what stays constant.

This is a consequential observation for the controller design, because it means the miss penalty is not an intrinsic property of position-independent reuse. It is a property of storing synchronously on the critical path. Moving the offload to a background stream, so that a request pays only for what it reads and not for what it writes, would remove the penalty for the 80% of our trace that misses. We did not implement this change, and validating it is the highest-value next step identified by this evaluation.

What Length-Based Routing Actually Buys

The crossover suggests a floor. In aggregate, however, the floor is not the rule that matters. Figure 4.10 compares three policies on the same trace: routing everything through the cache, applying a 2,000-token floor, and applying a band that adds an upper threshold at 6,000 tokens. We measure each by the aggregate deficit, the extra wall-clock time the cache path costs across the whole trace relative to serving every request on vanilla vLLM. Serving the entire trace on vanilla vLLM takes 726 s of TTFT in total, which is the reference point for these figures.

Routing everything through LMCache costs 772 s more than that. A 2,000-token floor barely moves it, to 748 s, even though it bypasses 149 of 536 requests. It recovers 3% of the deficit while giving up 28% of the traffic. Raising the floor does not rescue it either: a 5,000-token floor still leaves 698 s while bypassing half the workload. The reason is visible in the figure. The sub-breakeven mode is populated at every length, so removing short requests removes about as much benefit as cost, and the median speedup of what remains falls slightly rather than rising.

The upper threshold is what pays. An upper gate alone at 6,000 tokens, with no floor at all, brings the deficit from 772 s to 101 s. That single rule accounts for 87% of the total recovery available from any length-based policy we swept. Adding the 2,000-token floor on top brings it to 78 s. The mechanism is the one identified above: long prompts in this trace rarely hit, so they pay the full synchronous store cost with almost no compensating read. The upper gate is therefore better understood as a crude reuse gate than as a length gate. It works because length correlates with reuse at the top of this range, not because length is the quantity that matters.

Two limits are worth stating plainly. Every band we swept leaves the aggregate negative; the best of them, 5,000 to 6,000 tokens, still costs 27 s and does so by routing only 111 of 536 requests, which is narrow enough that it is really just bracketing this workload's dominant prompt size rather than expressing a general rule. And under every policy the median speedup of the retained population stays at or below 0.53×, because the retained set remains dominated by requests that miss. Length cannot separate hits from misses, so no threshold on length can make the cache path profitable here.

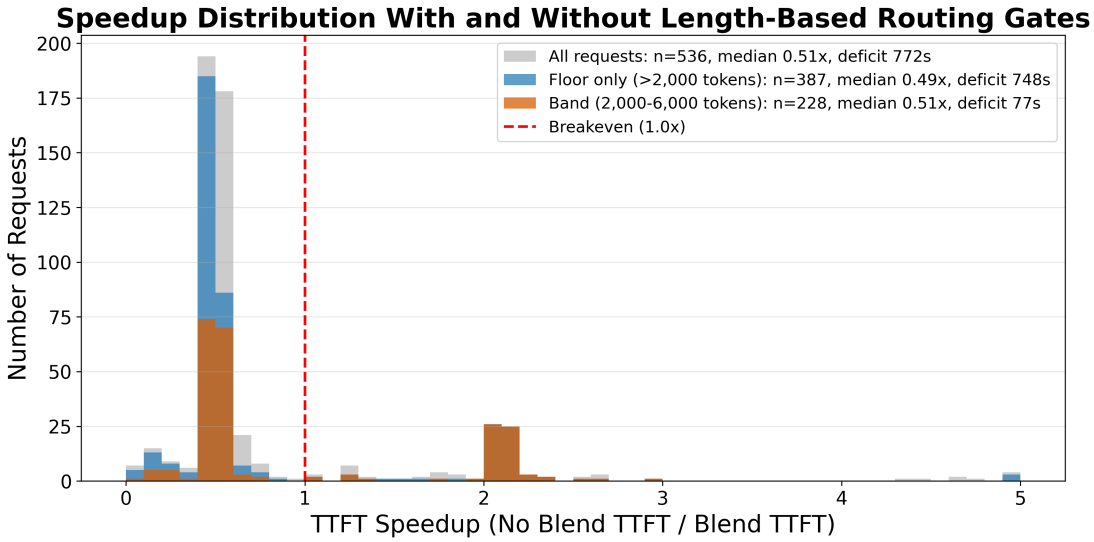

Figure 4.10: Per-request TTFT speedup under three routing policies. Bypassed requests are served by vanilla vLLM and sit at exactly 1.0× by construction, so they are excluded and each histogram shows only the requests still taking the cache path. The legend reports the aggregate deficit for each policy.

Proposed Controller

Taken together, these results argue for treating the LMCache layer as optional per request, mediated by a controller in front of the backend that applies two independent rules.

The first is a length band rather than a floor. Prompt length is known before dispatch, so the rule is cheap to evaluate, and the upper threshold is where nearly all of its value lies. The floor contributes little on its own and is best justified as protection against the short requests that cannot win even when they hit.

The second rule concerns reuse, and it is the one that determines whether the system is profitable overall. Since the outcome is governed by whether a request will hit, the controller needs an estimate of hit likelihood before it commits to the cache path. Both rules govern only *whether* a request takes the cache path, never what the model receives, so neither can affect the quality of the generated patch. Section 4.2 verifies the corresponding property of the cache path itself: that reusing KV state across positions leaves output quality intact.

Next Steps

Three concrete steps follow from this case study, in decreasing order of expected value.

Move the KV store off the critical path. Section 4.1.4 attributes the roughly 2× miss penalty to the synchronous offload, which the roughly 80% of requests that miss pay in full. Making the offload asynchronous would remove that penalty without changing the reuse mechanism at all, and would make the routing question far less pressing.

Build and validate a reuse predictor. A cheap approximation is available in the agentic setting: the frontend knows which context objects it assembled into the prompt, so it can report how many of them it has already sent. That estimate can be attached to the request

and used to route. Validating it requires a trace that samples hit ratio continuously, which ours does not, as noted in Section 5.3.

Deploy the length band as an interim measure. Until the store path is asynchronous and a predictor exists, an upper threshold near 6,000 tokens recovers most of what length-based routing can recover on this workload. A deployment that cannot predict reuse at all is better served by enabling LMCache selectively, for workloads known to have high context overlap, rather than by default.

4.2 Accuracy Evaluation for Context Retrieval

The preceding section treats latency as the only quantity at stake in the routing decision. That is only justified if cache reuse leaves the generated patches unchanged in quality, which this section establishes. It evaluates the frontend context retrieval algorithm (RepoGraph + Agentless) along two dimensions. First, we measure the resolve rate of the integrated agent across several LLM backends of different parameter sizes and architectures, to verify that the frontend behaves consistently when paired with different models. Second, we measure the resolve rate under different CacheBlend blending granularities, which is the check that licenses treating the cache path as a pure latency trade-off.

4.2.1 Setup

The evaluation of RepoGraph largely follows the methods of the original paper, where it is utilized as a plug-in component on top of an existing agentic framework. For the purpose of our study, we chose the Agentless procedural framework. The advantage of integrating a procedural framework in lieu of a traditional agent is simplicity and comprehensibility. Agentless deploys a three-phase process of localization, repair, and patch validation [Xia+24]. Intermediate results are transparent between each step, where empty or error responses are immediately identifiable. This allows for finding bottlenecks and comparing performance across various models at step granularity.

Our goal is to verify the consistency of the integrated agent's accuracy across LLM backends of different parameter sizes and architectures, and to verify that the backend cache reuse does not degrade output quality. For this purpose, we conducted experiments for RepoGraph + Agentless using a set of models with different configurations (for details see Models).

Metric

We adopt resolve rate following the original work as our primary evaluation metric. An instance is considered resolved if (1) the generated patch is successfully applied to the repository and (2) the patched code passes all test suites.

This metric provides a direct measure of end-to-end task success, capturing not only whether a patch is syntactically correct but also whether it integrates cleanly into the repository and satisfies the intended functionality. Compared to intermediate measures such as code similarity or partial compilation success, the resolve rate reflects the practical utility of the generated patches and therefore serves as a good indicator of real-world performance.

Datasets

For a comprehensive yet time- and cost-efficient evaluation, we run our experiments on SWE-bench Verified-mini [Mar25], which is a subset of SWE-bench Verified [Cho+24] selected via k-means clustering and linear programming so that the marginal distributions

of important scores are preserved. This is a smaller dataset than SWE-bench Lite [Yan+24], yet high-quality and well-specified, and is capable of reflecting an agent's performance across a spectrum of task difficulties.

Models

To ensure consistency, robustness, and portability of RepoGraph as the repository-aware context retrieval algorithm, it is important to evaluate it against multiple model backends of varying architectures and parameter scales. Different models exhibit distinct behaviors in terms of tokenization, embedding spaces, and context utilization. A retrieval method that aligns well with one tokenizer or attention mechanism may underperform when paired with another, which makes cross-model testing critical for avoiding over-specialization.

In addition, models differ in how they exploit the retrieved context. More powerful models may compensate for imperfect retrieval given stronger reasoning and synthesis capabilities, whereas smaller models are more sensitive to the accuracy and relevance of the retrieved context. Evaluating across a spectrum of models provides insight into how much of RepoGraph's accuracy benefits stem from the context retrieval strategy itself. This also allows us to characterize situations where retrieval quality is most impactful, particularly in cost-constrained settings where smaller models are attractive.

Finally, cross-backend evaluation provides practical benefits for deployment. Since real-world systems often migrate between providers (e.g., OpenAI, Anthropic, Mistral), retrieval algorithms must remain portable to ensure stable performance under backend substitution. For our experiments, we therefore consider a representative set of both proprietary and open-source models, including **GPT-4o, o4-mini, Claude 3.5 Sonnet, and Devstral Small**, covering a range of architectures, parameter counts, and costs.

4.2.2 Results

Figure 4.11 shows the resolve rate distribution for RepoGraph + Agentless across the four model backends on SWE-bench Verified-mini. The integrated agent achieves comparable resolve rates across the proprietary and open-source models considered, which supports the claim that RepoGraph generalizes across backends rather than being tuned to a specific model family. The Devstral Small results are particularly relevant for the integrated system evaluation, since Devstral Small is the open-source model used as the LLM backend in the latency evaluation (Section 4.1).

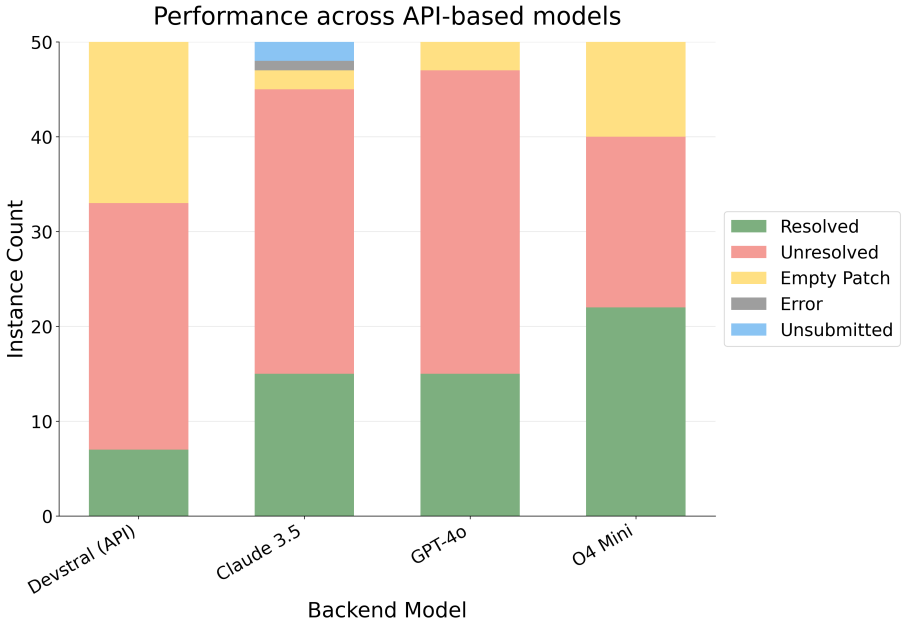

Figure 4.11: Resolve rate distributions for SWE-bench Verified-mini evaluation runs across different models.

To verify that CacheBlend's KV cache reuse does not affect output quality, we run an additional evaluation using Devstral Small hosted on vLLM with LMCache (CacheBlend enabled) on the same SWE-bench Verified-mini dataset. We compare different blending granularities against the baseline (no blending) to verify that position-independent cache reuse does not degrade patch quality. Figure 4.12 shows that the resolve rates across blending granularities are comparable to the no-blending baseline, confirming that the backend cache reuse preserves output quality even at finer chunk granularities.

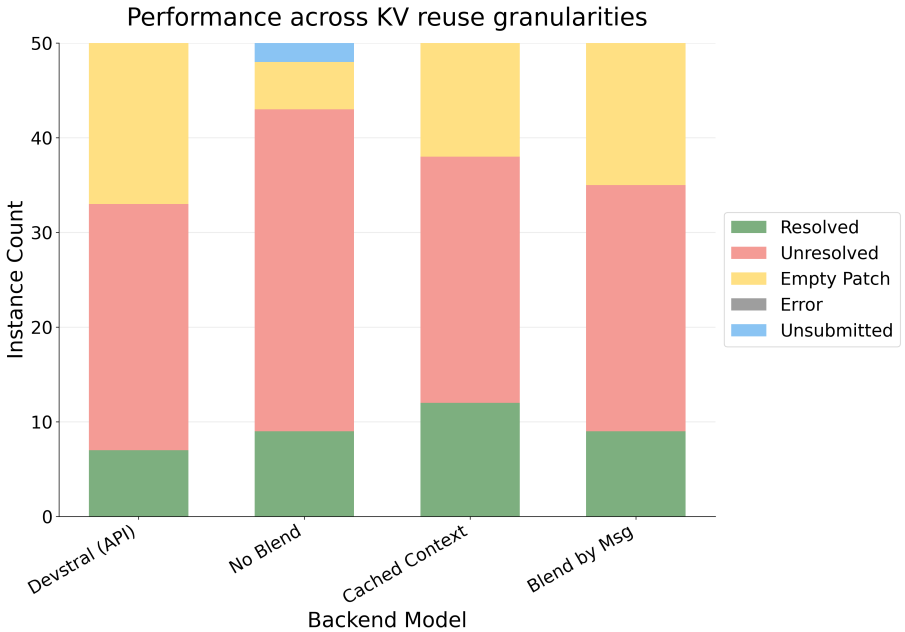

Figure 4.12: Resolve rate distributions for SWE-bench Verified-mini evaluation runs across different blending methods, served model is Devstral Small.

Chapter 5

Conclusion

This thesis investigated the latency behavior of agentic LLM serving under position-independent KV cache reuse, delivering a set of backend enhancements to LMCache, the reference implementation of CacheBlend, together with an evaluation that localizes the bottleneck that remains. The starting observation was that agentic prompts differ from the multi-round QA workloads that CacheBlend was originally evaluated on: they contain code blocks that are reordered or partially substituted across successive requests, arrive at higher concurrency, and span a far wider range of cache hit ratios. The upstream implementation leaves four aspects of this regime unaddressed, and this work delivered a corresponding set of fixes.

5.1 Summary of Contributions

The four backend enhancements are as follows. First, we added the `old_positions` field to `MemoryObjMetadata` together with its serialization handlers, populated it at save time, and connected it to the layerwise GPU connector's rotary embedding correction path. This completes the runtime handling for reusing a cached chunk at a position different from where it was originally cached, which is a common pattern in agentic workflows. Second, we introduced a `batched_allocate_varied` method in the CPU storage backend that acquires the buffer lock once per request rather than once per chunk, reducing lock contention proportionally to the number of chunks. Third, we bounded the eviction loop with a configurable retry limit (`MAX_EVICTION_RETRIES`), which prevents system-wide stalls when all cache entries are pinned by concurrent lookups. Fourth, we implemented a per-request profiling module that instruments the critical path with per-operation and per-phase timing records, enabling fine-grained latency attribution that would otherwise be invisible in aggregate statistics.

These enhancements were integrated into an end-to-end system in which the RepoGraph + Agentless frontend generates realistic agentic prompts, an adapter module bridges the frontend prompt format to the backend cache engine, and the backend serves the requests with CacheBlend enabled. A request logging and replay mechanism was added so

that the same trace can drive the backend under both blend and no-blend configurations, isolating the backend effect from agent-side variability.

5.2 Summary of Findings

The latency evaluation on a 537-request SWE-bench Verified-mini trace produced four main findings.

First, cache hit ratio, not prompt length, determines whether blending pays off. The trace splits into a low-hit cluster (hit ratio at most 34.2%, $n = 430$, median speedup 0.496) in which 2.1% of requests beat the baseline, and a high-hit cluster (hit ratio at least 72.3%, $n = 106$, median speedup 2.058) in which 81.1% do. No request in the trace falls between these two hit-ratio bands, so the breakeven crossing can be bounded but not measured.

Second, applied to all traffic, the cache layer is a net cost on this workload. Only 17.7% of requests were faster under blending, the median speedup is 0.509×, and summed over the trace the blend configuration spends 772 s more than vanilla vLLM would have.

Third, the reason is that the KV store runs synchronously inside the model execution step, before the first token is produced, and transfers a volume linear in prompt length. A request therefore pays for what it writes as well as what it reads, and because that cost grows in proportion to the baseline, the penalty for a cache miss stays near 0.5× at every prompt length from under 1,000 to over 12,000 tokens. Prompt length does not rescue it.

Fourth, short requests exhibit far higher speedup variance than long ones. This variance is driven by an interaction between vLLM's chunked-prefill scheduler and LMCache's per-step invocation pattern: requests that arrive during decode-heavy scheduling windows are fragmented across many prefill steps, and each step pays the per-call retrieve overhead of 30.8 ms in blend mode versus 18.0 ms without blending, of which GPU onload is 91%. This mechanism explains the spread among short requests but not the aggregate deficit, which the store path accounts for.

The frontend evaluation, run as a supporting cross-check, confirmed two properties of the integrated system. RepoGraph produces comparable resolve rates across four LLM backends (GPT-4o, o4-mini, Claude 3.5 Sonnet, Devstral Small), which supports its portability claim. Enabling CacheBlend at various blending granularities on Devstral Small yields resolve rates comparable to the no-blend baseline, which confirms that the position correction enhancement preserves output quality end to end.

5.3 Limitations and Future Work

Several limitations of the current evaluation suggest concrete next steps. The most important is that the SWE-bench Verified-mini trace samples cache hit ratio very unevenly. Most requests share little content with their predecessors, and no request at all falls in the 34.2% to 72.3% hit-ratio band, which is precisely where the breakeven crossing lies. A workload constructed to sweep hit ratio continuously would locate that crossing and turn the bound reported here into a measurement. This matters directly for the mitigation strategies below, since a bypass policy needs to know where the crossing is. The

bimodal fragmentation observed in the trace is best interpreted as an emergent property of steady-state scheduler load rather than as a hardcoded system constant. Running the same benchmark under different concurrency levels would characterize how the mode shifts and would generalize the fragmentation discussion.

The evaluation also points to a specific implementation change that was not made here. Section 4.1.4 traces the roughly 2× TTFT penalty on cache misses to the fact that the KV store runs synchronously inside the model execution step, before the first token is produced, and transfers a volume linear in prompt length. Because roughly 80% of the requests in our trace miss, this single property accounts for most of the aggregate slowdown we measured. Moving the offload off the critical path, so that a request pays for what it reads but not for what it writes, is the highest-value next step this work identifies. Alongside it, three mitigations for the small-request variance were identified but not implemented: a minimum token-per-step threshold for micro-batched steps, the prompt-length bypass evaluated in Section 4.1.4, and a coalesced retrieval mode that batches token ranges across prefill steps.

Beyond these targeted mitigations, the frontend interface is intentionally narrow, and swapping RepoGraph + Agentless for a more recent context retrieval module (for example, one based on agentic search rather than static graph queries) would provide a useful comparison point and would validate the pluggable frontend claim in practice.

Taken together, the results show that position-independent reuse is profitable for agentic workloads in the high-hit-ratio regime, but that applying it to all traffic is a net cost until the KV store is moved off the critical path. Prompt length turns out to be a poor proxy for that regime: it separates the profitable requests from the unprofitable ones only weakly, and the length-based routing rules we evaluated reduce the aggregate cost without eliminating it. The mechanism itself is sound, and the enhancements delivered here make it correct and diagnosable under agentic conditions. What the evaluation adds is a specific account of where the remaining cost lives and what it would take to remove it, which is the necessary step between a promising QA-workload benchmark and a production-ready component in a code agent.

Bibliography

This bibliography contains 21 references.

[Ant+25] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. *SWE-Search: Enhancing Software Agents with Monte Carlo Tree Search and Iterative Refinement*. 2025. arXiv: 2410.20285 [cs.AI]. URL: https://arxiv.org/abs/2410.20285.

[Ant25] Anthropic. *Introducing Claude 4*. Announces Claude Code general availability. May 2025. URL: https://www.anthropic.com/news/claude-4.

[Bai+23] Ramakrishna Bairi, Atharv Sonwane, Aditya Kanade, Vageesh D C, Arun Iyer, Suresh Parthasarathy, Sriram Rajamani, B. Ashok, and Shashank Shet. *CodePlan: Repository-level Coding using LLMs and Planning*. 2023. arXiv: 2309.12499 [cs.SE]. URL: https://arxiv.org/abs/2309.12499.

[Cho+24] Neil Chowdhury et al. *Introducing SWE-bench Verified*. 2024. URL: https://openai.com/index/introducing-swe-bench-verified/.

[Gim+24] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. *Prompt Cache: Modular Attention Reuse for Low-Latency Inference*. 2024. arXiv: 2311.04934 [cs.CL]. URL: https://arxiv.org/abs/2311.04934.

[Ho+20] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. *Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps*. 2020. arXiv: 2011.01060 [cs.CL]. URL: https://arxiv.org/abs/2011.01060.

[Jia+25] Zhonghao Jiang, Xiaoxue Ren, Meng Yan, Wei Jiang, Yong Li, and Zhongxin Liu. *CoSIL: Software Issue Localization via LLM-Driven Code Repository Graph Searching*. 2025. arXiv: 2503.22424 [cs.SE]. URL: https://arxiv.org/abs/2503.22424.

[Kwo+23] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. *Efficient Memory Management for Large Language Model Serving with PagedAttention*. 2023. arXiv: 2309.06180 [cs.LG]. URL: https://arxiv.org/abs/2309.06180.

[LIB25] Lukas, Ian, and Balta. *Cursor Agent CLI*. https://cursor.com/en/blog/cli. Cursor Blog. Aug. 2025.

[Mar25] MariusHobbhahn. *SWE-bench-verified-mini*. 2025. URL: https://huggingface.co/datasets/MariusHobbhahn/swe-bench-verified-mini.

[MS25] Taylor Mullen and Ryan J. Salva. *Introducing Gemini CLI: your open-source AI agent*. Google Developers Blog. 2025.

[Ouy+25] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. *RepoGraph: Enhancing AI Software Engineering with Repository-level Code Graph*. 2025. arXiv: 2410.14684 [cs.SE]. URL: https://arxiv.org/abs/2410.14684.

[SKS25] Nick Sharma, Mahi Kolla, and Ilai Soloducho. *Fully Reimagined: AI-First Google Colab*. Google Developers Blog. May 2025. URL: https://developers.googleblog.com/en/fully-reimagined-ai-first-google-colab/.

[Tri+22] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. *MuSiQue: Multihop Questions via Single-hop Question Composition*. 2022. arXiv: 2108.00573 [cs.CL]. URL: https://arxiv.org/abs/2108.00573.

[Xia+24] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. *Agentless: Demystifying LLM-based Software Engineering Agents*. 2024. arXiv: 2407.01489 [cs.SE]. URL: https://arxiv.org/abs/2407.01489.

[Yan+24] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. 2024. arXiv: 2405.15793 [cs.SE]. URL: https://arxiv.org/abs/2405.15793.

[Yao+25] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. *CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion*. 2025. arXiv: 2405.16444 [cs.LG]. URL: https://arxiv.org/abs/2405.16444.

[Zha+24a] Lei Zhang, Yunshui Li, Jiaming Li, Xiaobo Xia, Jiaxi Yang, Run Luo, Minzheng Wang, Longze Chen, Junhao Liu, and Min Yang. *Hierarchical Context Pruning: Optimizing Real-World Code Completion with Repository-Level Pretrained Code LLMs*. 2024. arXiv: 2406.18294 [cs.CL]. URL: https://arxiv.org/abs/2406.18294.

[Zha+24b] Xinrong Zhang et al. *∞Bench: Extending Long Context Evaluation Beyond 100K Tokens*. 2024. arXiv: 2402.13718 [cs.CL]. URL: https://arxiv.org/abs/2402.13718.

[Zha+24c] Yuntong Zhang, Haifeng Ruan, Zhiyu Fan, and Abhik Roychoudhury. *AutoCodeRover: Autonomous Program Improvement*. 2024. arXiv: 2404.05427 [cs.SE]. URL: https://arxiv.org/abs/2404.05427.

[Zhe+24] Lianmin Zheng et al. *SGLang: Efficient Execution of Structured Language Model Programs*. 2024. arXiv: 2312.07104 [cs.AI]. URL: https://arxiv.org/abs/2312.07104.